Tymbolica

Exact symbolic computation inside Typst

User manual · version 0.1.0

Parse Typst mathematics, transform it with Symbolica's exact algebra engine, and place the result directly back into a document.

Repository · MIT license · Symbolica guide · Symbolica license

Contents

1 Start here

Tymbolica keeps symbolic calculation beside the mathematics it belongs to. You can write a formula in Typst, factor or differentiate it, and place the answer straight back into the page—without copying expressions to another program.

Here is the whole pattern: read some mathematics with `math`, do the algebra, and display the result with `to-typst`.

```
#import "@local/tymbolica:0.1.0": *

#let p = math($(x + y)^3 - (x^3 + y^3)$)
#to-typst(factor(expand(p)))
```

$$3xy(x + y)$$

Tymbolica finds $3xy(x + y)$ exactly. The same three-step pattern—read, calculate, display—runs through the rest of this manual.

1.1 Installation

Until Tymbolica is published in Typst Universe, install it from a checkout. On Linux, place or symlink the repository root at:

`~/.local/share/typst/packages/local/tymbolica/0.1.0`

Use the corresponding Typst data directory on macOS or Windows. The package root must contain `typst.toml`; its `typst` directory contains `lib.typ`, the small `tymbolica.wasm` plugin, and `tymbolica-full.wasm` with Rubi integration. Then import:

```
#import "@local/tymbolica:0.1.0": *
```

From a source checkout, a document can instead import the library by relative path:

```
#import "path/to/tymbolica/typst/lib.typ": *
```

No build step is needed to use the package: everything required is already included in the checkout.

> **Why `@local`?** The examples use `@local` because this version is installed from the repository. A future Typst Universe release will use its published namespace instead.

1.2 Choose core or full

Most documents only need Tymbolica's algebra, solving, evaluation, and matrix tools. Those live in the small core plugin, which is selected by default. The much larger Rubi rule collection lives in a separate full plugin, so a document that never integrates does not have to load it.

Profile	Create it with	Use it for
Core	`init()`	Everything except Rubi integration; this is the default.
Full	`init(profile: "full")`	`integrate`, `integrate-with-steps`, and all core operations.

Both files ship with the package, so the profile changes what Typst loads, not the size of the package download. If you assemble a core-only deployment by hand, `tymbolica-full.wasm` can simply be left behind.

For an integration-heavy document, give the full engine a short name and use its methods explicitly:

```
#let sym = init(profile: "full")
#let parse = sym.math
#let var = sym.var
#let integrate = sym.integrate
#let x = var("x")
#let result = integrate(parse($x / (x + 1)$), x)
```

The imported top-level API is core-only, so integration always starts with an explicit full engine. Keep parsing, transformation, and rendering on that same engine: opaque Atom bytes cannot be passed between the core and full plugins.

1.3 Where to begin

If you want to…	Start with…
Turn Typst mathematics into a symbolic expression	`math`, `var`, and `atom`
Put an answer back into the document	`to-typst`
Factor, expand, differentiate, or take a series	`expand`, `factor`, `derivative`, `series` in the core profile
Integrate and inspect Rubi's rule path	`init(profile: "full")`, then `sym.integrate-with-steps`
Replace a recurring symbolic pattern	`wild`, `rule`, `replace`
Evaluate a formula at many points	`evaluate-many`, `evaluate-grid`
Solve equations exactly or numerically	`solve-linear`, `solve-system`, `nsolve-system`
Work with exact matrices	`matrix`, `matrix-solve`, `row-reduce`

2 A few ideas before the examples

2.1 Keep expressions symbolic until you display them

The value returned by `math` is a symbolic expression, not something Typst can typeset on its own. Pass it through as many algebraic operations as you like, then call `to-typst` where you want the answer to appear.

`Typst math` → `math` → `symbolic expression` → `transform` → `to-typst` → `document`

```
#let expression = math($(x + 1)^4$)
#let transformed = expand(expression)

original: $ #to-typst(expression) $\
expanded: $ #to-typst(transformed) $
```

original:

$$(1 + x)^4$$

expanded:

$$1 + 4x + 6x^2 + 4x^3 + x^4$$

For ordinary documents, `to-typst` is usually all you need. `to-latex` is handy when exporting to another system, while `canonical` shows Symbolica's plain text form when you are diagnosing a difficult expression.

2.2 Variables and wildcards have different jobs

Function	Typical input	Use
`math`	`$x^2 + 1$`	Read a Typst formula.
`atom`	a number, string, formula, or expression	Turn a general value into an expression.
`var`	`"x"`	Create a variable that belongs to the mathematics.
`wild`	`"a"`	Create a placeholder used only while matching a pattern.

> **The distinction that matters** A variable is part of the formula. A wildcard is a blank in a pattern. Use `var("x")` for the x in $x^2 + 1$; use `wild("a")` when a replacement rule should accept any expression in place of a.

Repeated occurrences of the same wildcard in one pattern must capture the same expression. Different matches can bind it differently; the rewriting guide below turns that rule into a concrete example.

2.3 Built-in functions such as sine and cosine

Polynomials work with the imported top-level functions. For analytic functions such as `sin`, `cos`, and `exp`, create a Symbolica-flavoured set of functions so that differentiation and numerical evaluation recognize them:

```
#let sym = init(namespace: "symbolica")
#let parse = sym.math
#let var = sym.var
#let derivative = sym.derivative
#let render = sym.to-typst
#let x = var("x")

#render(derivative(parse($sin(x)$), x))
```

The worked pendulum example binds these functions to short local names so that the calculation remains readable.

2.4 Write equations as expressions equal to zero

Solver inputs are expressions understood to equal zero. For example, `math($x + y - 3$)` represents $x + y = 3$.

For several variables, each solution row follows the variable order you give the solver. Exact solving finds all supported algebraic branches; numerical solving looks for one branch near your initial guess.

3 Worked mathematical guides

The quickest way to learn the package is to follow a calculation from question to answer. Each example below ends with a check, because a plausible-looking formula is not yet a convincing result.

3.1 Calibrate a pendulum model

A pendulum gives us a small but complete modelling problem. Begin with the potential $V(\theta) = \kappa(1 - \cos\theta)$, derive the restoring torque, and take a cubic small-angle approximation. Two torque readings are then enough to recover both the unknown scale κ and a sensor offset τ_0.

The calculation follows the pendulum-calibration example in Symbolica's repository README . We create the `symbolica` set of functions because `cos` must be understood as the analytic cosine rather than as an arbitrary function name.

3.1.1 Derive the model

```
#let sym = init(namespace: "symbolica")
#let (
  math: m, var: v, to-typst: render,
  derivative, series, neg, add,
) = sym

#let potential = m($kappa (1 - cos(theta))$)
#let q = v("θ")
#let k = v("κ")
#let b = v("τ₀")

#let torque = neg(derivative(potential, q))
#let small-angle = series(torque, q, 0, 3)
#let model = add(small-angle, b)

$ V(theta) = #render(potential) $\
$ tau(theta) = -(partial V)/(partial theta) = #render(torque) $\
$ tau_"small"(theta) + tau_0 = #render(model) $
```

$$V(\theta) = \kappa(1 - \cos(\theta))$$

$$\tau(\theta) = -\frac{\partial V}{\partial \theta} = -\kappa \sin(\theta)$$

$$\tau_{\text{small}(\theta)} + \tau_0 = -\kappa\theta + \frac{1}{6}\kappa\theta^3 + \tau_0$$

The derivative gives the restoring torque, while the cubic series keeps the first nonlinear correction to the familiar small-angle law. Adding τ_0 leaves a model that is linear in the two unknown parameters.

3.1.2 Fit two readings exactly

```
#let q1 = v("θ₁")
#let q2 = v("θ₂")
#let t1 = v("τ₁")
#let t2 = v("τ₂")

#let fit = solve-linear((
  sub(replace(model, q, q1), t1),
  sub(replace(model, q, q2), t2),
), (k, b))

$ kappa = #render(fit.at(0)) $\
$ tau_0 = #render(fit.at(1)) $
```

$$\kappa = \frac{6\ \tau_1 - 6\ \tau_2}{-6\ \theta_1 + 6\ \theta_2 + \theta_1^3 - \theta_2^3}$$

$$\tau_0 = \frac{-6\ \theta_1\ \tau_2 + 6\ \theta_2\ \tau_1 + \theta_1^3\ \tau_2 - \theta_2^3\ \tau_1}{-6\ \theta_1 + 6\ \theta_2 + \theta_1^3 - \theta_2^3}$$

Nothing has been rounded: both fitted parameters are still formulas in the angles and measured torques. Now insert two observations, $(\theta_1, \tau_1) = (0.10, -0.4697)$ and $(\theta_2, \tau_2) = (0.20, -0.9545)$.

3.1.3 Insert the measurements and check the fit

```
#let fitted = evaluate-many(
  fit,
  (q1, q2, t1, t2),
  ((0.10, 0.20, -0.4697, -0.9545),),
).first()
#let predictions = evaluate-many(
  model,
  (q, k, b),
  (
    (0.10, fitted.at(0).re, fitted.at(1).re),
    (0.20, fitted.at(0).re, fitted.at(1).re),
  ),
)

$ kappa approx #calc.round(fitted.at(0).re, digits: 6), quad
  tau_0 approx #calc.round(fitted.at(1).re, digits: 6) $\
predicted torques:
#predictions.map(row => str(calc.round(row.first().re, digits: 4))).join(", ")
```

$$\kappa \approx 4.905228, \quad \tau_0 \approx 0.020005$$

```
predicted torques: -0.4697, -0.9545
```

The formula for κ remains exact until the measured values are inserted. Substituting the fitted parameters back into the model reproduces both torque readings, so the final line checks the entire chain from potential to fit.

3.2 Rewrite a repeated identity with wildcards

This is where a wildcard earns its keep. Suppose an expression contains several copies of $\sin^2(a) + \cos^2(a)$, but the argument a is different each time. The pattern should insist that the two functions share an argument without fixing what that argument is.

```
#let source = math(
  $3 (sin(x)^2 + cos(x)^2)
    + sin(x + y)^2 + cos(x + y)^2$
)
#let identity = math(
  $sin("a_")^2 + cos("a_")^2$
)

#let one-pass = replace(source, identity, 1)
#let reduced = replace(source, identity, 1, repeat: true)

source: #to-typst(source)\
one pass: #to-typst(one-pass)\
repeat to a fixed point: #to-typst(reduced)
```

`source:` $3(\sin(x)^2 + \cos(x)^2) + \sin(x+y)^2 + \cos(x+y)^2$
`one pass:` $1 + 3(\sin(x)^2 + \cos(x)^2)$
`repeat to a fixed point:` 4

Within one match, both occurrences of `a_` capture the same expression. Across matches it first captures one argument and then another. This is the practical difference between an ordinary `var("a")` and `wild("a")`.

> **Repeating rules** `repeat: true` keeps applying the rule until the expression stops changing. Use it only for rules that settle: $a \to a + 1$, for example, would never reach a fixed point.

3.3 Follow Rubi's integration steps

For $\frac{x}{1+x}$, the useful idea is to expose a constant term before integrating. Rubi finds that rewrite, splits the resulting integral, and records the nested rule path. This is the same example used in Symbolica's integration introduction. Here the full engine is explicit because the core plugin deliberately omits Rubi.

```
#let sym = init(profile: "full")
#let parse = sym.math
#let var = sym.var
#let render = sym.to-typst
#let integrate-with-steps = sym.integrate-with-steps
#let derivative = sym.derivative
#let subtract = sym.sub
#let together = sym.together
#let x = var("x")
#let f = parse($x / (x + 1)$)
#let integration = integrate-with-steps(f, x)
#let residual = together(
  subtract(derivative(integration.result, x), f)
)
#assert(integration.complete)

$ f(x) = #render(f) $\
#for step in integration.steps [
  #h(step.depth * 1.25em)
  #if step.rule == none [*Transformation*] else [*Rule #step.rule*]
  #if step.description != "" [: #step.description]
  #linebreak()
  #h(step.depth * 1.25em)
  $#render(step.input) = #render(step.output)$
  #linebreak()
]
$ integral f(x) dif x = #render(integration.result) + C $\
verification: $ (partial I)/(partial x) - f(x) = #render(residual) $
```

$$f(x) = \frac{x}{1+x}$$

Rule 49 : Algebraic expansion
$\int \frac{x}{1+x}\,\mathrm{d}x = \int 1 + \frac{-1}{1+x}\,\mathrm{d}x$
  Rule 2009 : Integrate the terms of the sum separately.
  $\int 1 + \frac{-1}{1+x}\,\mathrm{d}x = \int 1\,\mathrm{d}x + \int \frac{1}{-1-x}\,\mathrm{d}x$
    Rule 24 : Apply the direct antiderivative formula.
    $\int 1\,\mathrm{d}x = x$
    Rule 16 : Apply the direct antiderivative formula.
    $\int \frac{-1}{1+x}\,\mathrm{d}x = -\log(1+x)$

$$\int f(x)\,\mathrm{d}x = x - \log(1+x) + C$$

verification:

$$\frac{\partial I}{\partial x} - f(x) = 0$$

The mathematics is visible in the tree: first $\frac{x}{1+x} = 1 - \frac{1}{1+x}$, then linearity separates the two integrals, and the leaves give x and $-\log(1+x)$. The indentation comes from each step's `depth`, so an outer rewrite is followed by the subintegrals it created. For a numbered rule, `input` and `output` are the integral before and after that rewrite. A step without a rule number records an auxiliary transformation, such as a fresh-symbol substitution. `description`, `rule`, `references`, and `source` explain where each move came from. The displayed result adds the customary $+C$; Tymbolica itself does not.

Best-effort integrals Rubi covers many families of integrands, but no finite rule collection solves every integral. When it stops early, `complete` is `false`, `result` contains an `unintegrable` marker for the unresolved part, and `steps` still records the progress.

3.4 Separate a rational response into modes

Suppose a response function arrives in a form with one removable factor and two remaining poles. Cancel the shared factor, split the reduced response into simple modes, and then recombine the modes to check that nothing was lost. This follows the rational-expression workflow in Symbolica's First Steps.

```
#let s = var("s")
#let response = math($((s + 3)(2 s + 5)) / (s^3 + 6 s^2 + 11 s + 6)$)
#let reduced = cancel(response)
#let modes = apart(reduced, s)
#let residual = together(sub(modes, reduced))

$
  H(s) &= #to-typst(response) \
  H_"reduced"(s) &= #to-typst(reduced) \
                 &= #to-typst(modes)
$\
verification: $ #to-typst(residual) $
```

$$H(s) = \frac{(3+s)(5+2s)}{6+11s+6s^2+s^3}$$
$$H_{\text{reduced}(s)} = \frac{5+2s}{2+3s+s^2}$$
$$= \frac{3}{1+s} + \frac{-1}{2+s}$$

verification:

$$0$$

The two terms expose poles at $s = -1$ and $s = -2$ separately, which is often the useful form for inverse transforms or modal reasoning. `together` returns a zero residual after recombination. The cancellation does hide the original restriction $s \neq -3$; keep that restriction when the domain matters.

3.5 Solve a nonlinear system exactly and numerically

The circle $x^2 + y^2 = 25$ and the line $x - y = 1$ meet twice. The exact solver should find both intersections; the numerical solver should find the one nearest its starting point.

```
#let x = var("x")
#let y = var("y")
#let system = (
  math($x^2 + y^2 - 25$),
  math($x - y - 1$),
)

#let exact = solve-system(system, (x, y))
#let positive = nsolve-system(
  system, (x, y), (3.0, 3.0), prec: 1e-10,
)
#let negative = nsolve-system(
  system, (x, y), (-3.0, -3.0), prec: 1e-10,
)
#let checks = evaluate-many(
  system, (x, y), (positive, negative),
)
```

```
exact branches:
1. x = 4, y = 3
2. x = -3, y = -4
seed (3, 3) → (4, 3); maximum residual 0.0
seed (-3, -3) → (-3, -4); maximum residual 3.552713678800501e-15
```

The exact calculation finds both branches. Starting near either intersection selects that numerical branch, and the small residual confirms that the point lies on both curves. A poor starting point may still converge elsewhere—or not at all.

3.6 Recover an interpolating polynomial with matrices

Find the quadratic through $(0, 1)$, $(1, 3)$, and $(2, 8)$. Writing $p(t) = a_0 + a_1 t + a_2 t^2$ turns interpolation into the exact matrix equation $Aa = b$.

```
#let t = var("t")
#let A = matrix((
  (1, 0, 0),
  (1, 1, 1),
  (1, 2, 4),
))
#let b = vec((1, 3, 8))
#let coefficients = matrix-solve(A, b)
#let check = matrix-mul(A, coefficients)
#let exact = matrix-is-zero(matrix-sub(check, b))
#let polynomial = add(
  matrix-at(coefficients, 0, 0),
  mul(matrix-at(coefficients, 1, 0), t),
  mul(matrix-at(coefficients, 2, 0), pow(t, 2)),
)

$ A = #to-typst(A), quad b = #to-typst(b) $\
$ op("det")(A) = #to-typst(det(A)) $\
$ a = #to-typst(coefficients) $\
$ p(t) = #to-typst(polynomial) $\
$ A a = #to-typst(check) $\
verification: residual matrix is exactly zero: #exact
```

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 3 \\ 8 \end{pmatrix}$$

$$\det(A) = 2$$

$$a = \begin{pmatrix} 1 \\ \frac{1}{2} \\ \frac{3}{2} \end{pmatrix}$$

$$p(t) = 1 + \frac{1}{2}t + \frac{3}{2}t^2$$

$$Aa = \begin{pmatrix} 1 \\ 3 \\ 8 \end{pmatrix}$$

```
verification: residual matrix is exactly zero: true
```

The nonzero determinant tells us the coefficients are unique. Reading them back gives the polynomial, and multiplying Aa recovers the three original measurements.

3.7 Map a gradient across a grid

Consider the quadratic surface $f(x, y) = x^2 + xy + y^2$. We first derive its two gradient components, then sample the height and gradient together on a small Cartesian grid.

```
#let x = var("x")
#let y = var("y")
#let f = math($x^2 + x y + y^2$)
#let fx = derivative(f, x)
#let fy = derivative(f, y)
#let grid = evaluate-grid(
  (f, fx, fy),
  (x, y),
  (
    domain(-1, 1, samples: 3),
    domain(-1, 1, samples: 3),
  ),
)

$ f = #to-typst(f) $\
$ partial_x f = #to-typst(fx), quad partial_y f = #to-typst(fy) $
```

$$f = xy + x^2 + y^2$$

$$\partial_x f = 2x + y, \quad \partial_y f = x + 2y$$

x	y	f	$\partial_x f$	$\partial_y f$
-1	-1	3	-3	-3
-1	0	1	-2	-1
-1	1	1	-1	1
0	-1	1	-1	-2
0	0	0	0	0
0	1	1	1	2
1	-1	1	1	-1
1	0	1	2	1
1	1	3	3	3

The table makes the geometry visible: the gradient vanishes at the origin and points uphill everywhere else. `evaluate-grid` pairs each point with one value for every requested expression; use `evaluate-many` when your sample points do not form a Cartesian product.

4 What to expect

Tymbolica deliberately presents a smaller surface than Symbolica itself. The parts covered in this manual work well for exact algebra in documents, but a few boundaries are worth knowing before you choose an approach:

- The imported top-level API and the default `init()` engine are core-only. Rubi integration is available from `init(profile: "full")`.
- Atom bytes belong to the plugin instance that created them. Bind the parser, operations, and renderer from one engine rather than mixing core and full calls in the same expression pipeline.
- Integration returns a best-effort expression when no Rubi rule finishes the job. Check `complete` from `integrate-with-steps` when an unevaluated remainder matters. No $+C$ is added automatically.
- Exact system solving is intended for linear and polynomial equations. Numerical solving depends on a starting point and gives an approximate answer rather than a derivation of convergence.
- Decimal literals remain floating-point values. A current upstream Wasm bug can mis-evaluate them inside analytic functions. Write exact fractions for those inputs—for example, `cos(1/2)`—and apply `to-float` to the result.
- Matrix entries must be rational-polynomial expressions, and their dimensions must agree for the requested operation.
- Algebraic transformations do not keep a separate list of assumptions. If a manipulation is valid only under a condition such as $x \neq 1$, preserve that condition in the surrounding document.
- Some unusual Typst math structures may not parse. `array-tree` can help show what the parser received.

For operations beyond this scope—an integral Rubi cannot finish, arbitrary precision, or deeper polynomial algorithms—use Symbolica directly.

4.1 When something looks wrong

Symptom	Likely cause	What to try
`expected content, found bytes`	A symbolic result was inserted directly into `$...$`.	Display it with `to-typst`.
An engine has no `integrate` method	It was created with the default core profile.	Use `init(profile: "full")` and bind its integration functions.

Symptom	Likely cause	What to try
A symbolic result fails in another API	The Atom bytes were created by a different plugin instance or profile.	Parse, transform, and render through functions bound from the same engine.
A derivative or series leaves a function unchanged	`sin`, `cos`, or another analytic function was read as an ordinary name.	Use `init(namespace: "symbolica")` for Symbolica built-ins.
An analytic function of a decimal gives an unexpected value	Its floating-point argument encountered the current upstream Wasm bug.	Use an exact fraction such as `cos(1/2)`, then call `to-float`.
A replacement does not match	Repeated wildcards would have to capture different expressions.	Check that every occurrence of the wildcard should match the same value.
An exact solver errors	An equation was not rearranged to zero, or the system is outside the supported polynomial scope.	Move every term to the left; try `nsolve-system` for a numerical branch.
A numerical solver errors or finds an unwanted root	The starting point leads to a different root or no root.	Try another physically meaningful initial guess and check residuals.
A matrix operation errors	Shapes differ, a matrix is singular, or an entry is unsupported.	Inspect `matrix-shape`, `det`, and the coefficient expressions.

5 API reference

The worked chapters are meant for reading; this section is meant for looking things up. The generated groups below are the core-only top-level API. The two full-profile integration methods are documented separately afterwards.

5.1 Parsing, symbols, and rendering

5.1.1 math

Parse Typst math content into an opaque Symbolica atom payload.

Arithmetic, fractions, powers, roots, absolute values, calls, and common Typst math structures are translated through the configured grammar. Matrix-valued `mat(...)` and `vec(...)` content must instead be passed to `matrix` or `vec`. Keep the returned bytes opaque and use this module's functions to inspect or transform them. Decimal literals remain floating-point coefficients; write a fraction when you need exact input.

```
#let expr = math($x + 1$)
#to-typst(expr)
```

$1 + x$

Parameters

```
math(
  eqn: content,
  grammar: dictionary none,
  namespace: str none
) -> bytes
```

eqn `content`

Math content, normally written as `$...$`.

grammar `dictionary` or `none`

Parser grammar override. `none` uses the default engine grammar.

Default: `none`

namespace `str` or `none`

Namespace for parsed symbols. `none` uses the engine namespace (`"typst"` for the top-level function).

Default: `none`

5.1.2 atom

Convert a supported Typst value or math expression into an atom payload.

Existing atom bytes pass through unchanged. Content is parsed like `math`; integers become exact numbers and floats remain floating-point coefficients. Strings are parsed as leaf values: numeric strings become numbers and other strings become symbols in the engine's namespace. Matrix payloads are not atom payloads and should not be passed to atom-only algebra functions.

```
#to-typst(atom("x"))
```

x

Parameters

```
atom(value: bytes content int float str) -> bytes
```

value `bytes` or `content` or `int` or `float` or `str`

Value to convert.

5.1.3 var

Construct a named Symbolica variable.

Unlike `wild`, this is an ordinary mathematical symbol and therefore does not capture subexpressions during pattern matching.

```
#to-typst(var("x"))
```

x

Parameters

```
var(
  name: str,
  namespace: str none
) -> bytes
```

name `str`

Symbol name without a namespace prefix or wildcard suffix.

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: `none`

5.1.4 wild

Construct a Symbolica pattern wildcard.

This creates the symbol `name` followed by `level` underscores. A wildcard is a pattern placeholder used by `rule`, `replace`, and `replace-wildcards`; it is not an ordinary unknown for algebra or solving. Use `var` for a mathematical variable.

```
#raw(canonical(wild("a")))
```

```
a_
```

Parameters

```
wild(
  name: str,
  level: int,
  namespace: str none
) -> bytes
```

name `str`

Base name of the wildcard, without trailing underscores.

level `int`

Number of underscore levels to append. `1` creates a conventional single wildcard such as `a_`; `0` appends no underscore and therefore creates an ordinary symbol instead.

Default: `1`

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: `none`

5.1.5 array-tree

Render the parse tree for a Typst math expression.

This is a diagnostic view of the tree consumed by `math`; it does not create a Symbolica atom.

```
#array-tree($(x + 1)^2$)
```

```
(
  strong(body: raw(text: "pow")),
  (
    styled(child: raw(text: "base"), ..),
    (
      strong(body: raw(text: "()")),
      (
        styled(child: raw(text: "expr"), ..),
        (
          strong(body: raw(text: "add")),
          equation(body: [x]),
          equation(body: [1]),
        ),
      ),
    ),
  ),
  (
    styled(child: raw(text: "exp"), ..),
    equation(body: [2]),
  ),
)
```

Parameters

```
array-tree(
  eqn: content,
  grammar: dictionary none
) -> content
```

eqn `content`

Math content to inspect.

grammar `dictionary` or `none`

Parser grammar override. `none` uses the engine grammar.

Default: `none`

5.1.6 canonical

Render an atom or matrix payload as Symbolica source text.

```
#raw(canonical(math($x + 1$)))
```

```
1+x
```

Parameters

```
canonical(
  expr: bytes content int float str,
  namespaces: bool
) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload. Other supported expression values are first converted as by `atom`.

namespaces `bool`

Include symbol namespaces in the output.

Default: `false`

5.1.7 to-typst-source

Render an atom or matrix payload as Typst math source.

```
#raw(to-typst-source(math($x + 1$)))
```

```
1+x
```

Parameters

```
to-typst-source(expr: bytes content int float str) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

5.1.8 to-typst

Render an atom or matrix payload as evaluated Typst math content.

The payload is first printed as Typst math source and then evaluated in math mode. Use `to-typst-source` when you need the source string instead.

```
#to-typst(math($x + 1$))
```

$1 + x$

Parameters

```
to-typst(
  expr: bytes content int float str,
  block: bool
) -> content
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

block `bool`

Render as a block equation instead of inline math.

Default: `false`

5.1.9 to-latex

Render an atom or matrix payload as LaTeX source.

```
#raw(to-latex(math($x^2 + 1$)))
```

```
1+x^{2}
```

Parameters

```
to-latex(expr: bytes content int float str) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

5.1.10 to-float

Approximate numerical coefficients and built-in functions as decimals.

Variables remain symbolic. The WebAssembly build supports between 1 and 16 significant decimal digits. Use `evaluate` on the original expression when you need a numerical value rather than a symbolic display form. Expressions containing built-in calls with very large numeric arguments remain exact.

```
#to-typst(to-float(math($1/3$), decimal-prec: 6))
```

$$0.333333$$

Exact rational arguments to Symbolica built-ins can be approximated too:

```
#let sym = init(namespace: "symbolica")
#let parse = sym.math
#let approximate = sym.to-float
#let render = sym.to-typst
#render(approximate(parse($cos(1/2)$), decimal-prec: 6))
```

$$0.877583$$

Parameters

```
to-float(
  expr: bytes content int float str,
  decimal-prec: int
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to approximate.

decimal-prec `int`

Number of significant decimal digits to retain, from 1 through 16.

Default: 16

5.2 Algebra and core calculus

5.2.1 simplify

Re-import and re-export an atom in Symbolica's canonical internal form.

This operation does not perform algebraic simplification; it currently acts as a normalization and validation round-trip for atom payloads.

```
#to-typst(simplify(math($x + 0$)))
```

$$x$$

Parameters

```
simplify(expr: bytes content int float str) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to normalize.

5.2.2 expand

Expand an expression through Symbolica's polynomial expansion.

```
#to-typst(expand(math($(x + 1)^2$)))
```

$1 + 2x + x^2$

Parameters

```
expand(expr: bytes content int float str) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to expand.

5.2.3 factor

Factor an expression exactly.

```
#to-typst(factor(math($x^2 + 2 x + 1$)))
```

$(1 + x)^2$

Parameters

```
factor(
  expr: bytes content int float str,
  complex: bool,
  square-free: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to factor.

complex `bool`

Factor over the complex rationals instead of the rationals. Cannot be combined with `square-free`.

Default: `false`

square-free `bool`

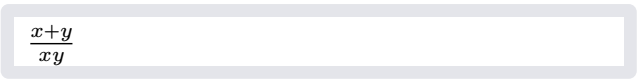
Return a square-free factorization, preserving multiplicities without fully splitting every factor. Complex input is handled over the complex rationals. Cannot be combined with `complex`.

Default: `false`

5.2.4 together

Write a rational expression over a common denominator.

```
#to-typst(together(math($1/x + 1/y$)))
```



Parameters

```
together(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

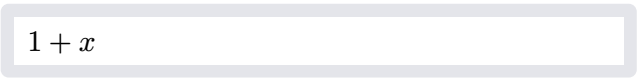
Expression whose rational terms should be combined.

5.2.5 cancel

Cancel common factors between numerators and denominators.

Parts of the expression without a cancellation are left alone. Remember that canceling a factor can hide a point excluded by the original formula.

```
#to-typst(cancel(math($(x^2 - 1)/(x - 1)$)))
```



Parameters

```
cancel(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

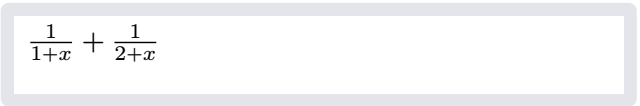
Rational expression in which to cancel common factors.

5.2.6 apart

Decompose a rational expression into partial fractions.

Denominators are decomposed in the given indeterminate.

```
#let x = var("x")
#to-typst(apart(math($(2 x + 3)/((x + 1)(x +
2))$), x))
```



Parameters

```
apart(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr bytes or content or int or float or str

Rational expression to decompose.

var bytes or content or str

Indeterminate for the decomposition.

5.2.7 collect

Collect terms by powers of one or more variables or functions.

```
#let x = var("x")
#to-typst(collect(math($5 x + x y + x^2 + 5$),
x))
```

$5 + x(5 + y) + x^2$

Parameters

```
collect(
  expr: bytes content int float str,
  variables: bytes content str array
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression whose terms should be collected.

variables `bytes` or `content` or `str` or `array`

One variable or function, or an array of them.

5.2.8 coefficient

Extract the coefficient of a literal monomial or subexpression.

For example, asking for the coefficient of `x^2` in $5x + xy + x^2 + yx^2$ returns $1 + y$.

```
#let x = var("x")
#to-typst(coefficient(math($5 x + x y + x^2 + y
x^2$), pow(x, 2)))
```

$1 + y$

Parameters

```
coefficient(
  expr: bytes content int float str,
  monomial: bytes content int float str
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

monomial `bytes` or `content` or `int` or `float` or `str`

Literal monomial or subexpression whose coefficient is wanted.

5.2.9 coefficient-list

Return collected key–coefficient pairs for one or more indeterminates.

Each result is `(key, coefficient)`, both as atom payloads. A key of `1` carries terms not polynomially collected in the requested variables. A coefficient that vanishes only through a deeper identity may remain.

```
#let x = var("x")
#let pairs = coefficient-list(math($x^2 + 5 x + 7$), x)
#pairs.map(pair => [#to-typst(pair.at(0)): #to-typst(pair.at(1))]).join[, ]
```

$1: 7,\ x: 5,\ x^2: 1$

Parameters

```
coefficient-list(
  expr: bytes content int float str,
  variables: bytes content str array
) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

variables `bytes` or `content` or `str` or `array`

One variable or function, or an array of them.

5.2.10 terms

Return the top-level additive terms of an expression.

This does not expand or recurse. A value that is not a sum is returned as a one-element array.

```
#terms(math($x^2 + 2 x + 1$)).map(to-typst).join[, ]
```

$1,\ 2x,\ x^2$

Parameters

```
terms(expr: bytes content int float str) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to split into top-level summands.

5.2.11 indeterminates

Return the variables and function expressions that act as indeterminates.

Results are sorted in Symbolica's internal order.

```
#indeterminates(math($f(x) + y$)).map(to-typst).join[, ]
```

$x,\ y,\ f(x)$

Parameters

```
indeterminates(
  expr: bytes content int float str,
  enter-functions: bool
) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

enter-functions `bool`

Traverse function arguments as well as collecting the function itself.

Default: `true`

5.2.12 contains

Test whether an expression literally contains another expression.

The test follows the stored expression structure: for example, `x*y*z` contains `x`, but does not contain the regrouped product `x*y` as a node.

```
#contains(math($x y z$), var("x"))
```

```
true
```

Parameters

```
contains(
  expr: bytes content int float str,
  subexpression: bytes content int float str
) -> bool
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to search.

subexpression `bytes` or `content` or `int` or `float` or `str`

Literal subexpression to look for.

5.2.13 is-constant

Test whether an expression has no user-defined variables or functions.

Supported built-in functions such as `sin` and `cos` remain constant when all of their arguments are constant.

```
#is-constant(math($cos(2) + 1/3$))
```

```
true
```

Parameters

```
is-constant(expr: bytes content int float str) -> bool
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to test.

5.2.14 derivative

Differentiate an expression exactly with respect to an indeterminate.

```
#let x = var("x")
#to-typst(derivative(math($(x + 1)^2$), x))
```

$2(1 + x)$

Parameters

```
derivative(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to differentiate.

var `bytes` or `content` or `str`

Symbolica indeterminate, normally created with `var`.

5.2.15 series

Compute a univariate series expansion around `expansion-point`.

The truncation depth is the rational number `depth / depth-denom`. With an absolute depth it is measured directly in `var`; with a relative depth it is measured from the lowest order encountered in the expression.

```
#let sym = init(namespace: "symbolica")
#let m = sym.math
#let v = sym.var
#let ser = sym.series
#let render = sym.to-typst
#render(ser(m($cos(x)/(x + 1)$), v("x"), 0, 3))
```

$1 - x + \frac{1}{2}x^2 - \frac{1}{2}x^3$

Parameters

```
series(
  expr: bytes content int float str,
  var: bytes content str,
  expansion-point: bytes content int float str,
  depth: int,
  depth-denom: int,
  depth-is-absolute: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to expand.

var `bytes` or `content` or `str`

Expansion variable, normally created with `var`.

expansion-point `bytes` or `content` or `int` or `float` or `str`

Point about which to expand.

depth `int`

Numerator of the requested truncation depth.

depth-denom `int`

Denominator of the truncation depth.

Default: `1`

depth-is-absolute `bool`

Use an absolute depth when `true`, or a depth relative to the lowest order in the expression when `false`.

Default: `true`

5.3 Rewriting

5.3.1 rule

Build a reusable replacement rule for `replace-multiple`.

`pattern` and `rhs` may contain symbols created with `wild`. The returned dictionary contains opaque atom payloads plus the matching options; pass it to `replace-multiple` rather than editing it manually.

```
#let r = rule(math($f("a_")$), math($g("a_")$))
#to-typst(replace-multiple(math($f(x)$), (r,)))
```

$g(x)$

Parameters

```
rule(
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
  level-range: array none,
  level-is-tree-depth: bool,
  partial: bool,
  allow-new-wildcards-on-rhs: bool,
  rhs-cache-size: int
) -> dictionary
```

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern to match.

rhs `bytes` or `content` or `int` or `float` or `str`

Expression substituted for each match. Wildcards captured by `pattern` are substituted in this expression.

non-greedy-wildcards `array`

Wildcards that should prefer the smallest possible match.

Default: `()`

min-level `int`

Lowest expression level at which matching is allowed; the root is level zero.

Default: `0`

max-level `int` or `none`

Highest allowed matching level, or `none` for no upper bound.

Default: `none`

level-range `array` or `none`

Optional `(minimum, maximum)` pair overriding `min-level` and `max-level`; the maximum may be `none`.

Default: `none`

level-is-tree-depth `bool`

Count full expression-tree depth when `true`; otherwise levels increase when entering functions.

Default: `false`

partial `bool`

Allow a pattern to match part of a sum, product, or other term instead of requiring the whole term.

Default: `true`

allow-new-wildcards-on-rhs `bool`

Permit wildcards on `rhs` that do not occur in `pattern`. When `false`, such rules are rejected.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; use zero to disable this cache.

Default: `100`

5.3.2 replace

Replace subexpressions matching `pattern` with `rhs`.

By default all non-overlapping outermost matches are replaced once. `repeat: true` reapplies the rule until the expression stops changing; rules that cycle therefore do not terminate. Use `rule` plus `replace-multiple` when several patterns must be considered together.

```
#to-typst(replace(math($f(x, y)$),
math($f("a_", "b_")$), math($g("b_", "a_")$)))
```

$g(y, x)$

Parameters

```
replace(
  expr: bytes content int float str,
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
  level-range: array none,
  level-is-tree-depth: bool,
  partial: bool,
  allow-new-wildcards-on-rhs: bool,
  rhs-cache-size: int
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression in which to replace matches.

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern to match.

rhs `bytes` or `content` or `int` or `float` or `str`

Replacement expression.

repeat `bool`

Reapply the rule until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from the deepest outward, acting on the result of each inner replacement.

Default: `false`

non-greedy-wildcards `array`

Wildcards that should prefer the smallest possible match.

Default: `()`

min-level `int`

Lowest allowed matching level; the root is level zero.

Default: `0`

max-level `int` or `none`

Highest allowed matching level, or `none` for no upper bound.

Default: `none`

level-range `array` or `none`

Optional `(minimum, maximum)` pair overriding `min-level` and `max-level`.

Default: `none`

level-is-tree-depth `bool`

Count full tree depth instead of function-entry depth.

Default: `false`

partial `bool`

Allow matching a part of a term rather than the entire term.

Default: `true`

allow-new-wildcards-on-rhs `bool`

Permit `rhs` wildcards that are absent from `pattern`.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; zero disables the cache.

Default: `100`

5.3.3 replace-multiple

Apply several reusable replacement rules together.

The traversal options apply to the combined rule set. With `repeat: true`, the complete set is reapplied until no rule changes the expression; cyclic rule sets do not terminate.

```
#let r1 = rule(math($f("a_")$),
math($h("a_")$))
#let r2 = rule(var("x"), var("z"))
#to-typst(replace-multiple(math($f(x) + x$),
(r1, r2)))
```

$$z + h(x)$$

Parameters

```
replace-multiple(
  expr: bytes content int float str,
  rules: array,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression in which to replace matches.

rules `array`

Array of dictionaries returned by `rule`.

repeat `bool`

Reapply the rule set until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from deepest to outermost, acting on intermediate results.

Default: `false`

5.3.4 replace-wildcards

Substitute explicit values for wildcard placeholders in a pattern.

This does not search another expression. It transforms `pattern` itself and requires each key in `replacements` to be a wildcard symbol.

```
#to-typst(replace-wildcards(math($k("a_")$),
((wild("a"), math($x + 1$)),)))
```

$k(1 + x)$

Parameters

```
replace-wildcards(
  pattern: bytes content int float str,
  replacements: array
) -> bytes
```

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern containing wildcards to substitute.

replacements `array`

Array of `(wildcard, replacement)` pairs.

5.4 Evaluation and solving

5.4.1 evaluate

Evaluate one expression numerically with optional substitutions.

`values` maps atom keys to real numbers or complex dictionaries of the form `(re: number, im: number)`. Evaluation must eliminate every unsupported symbolic quantity. The result always has the exact shape `(re: float, im: float)`, even when it is real.

```
#let x = var("x")
#let y = var("y")
#repr(evaluate(math($x^2 + y$), values: ((x,
2.0), (y, 3.0))))
```

```
(re: 7.0, im: 0.0)
```

Parameters

```
evaluate(
  expr: bytes content int float str,
  values: array
) -> dictionary
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to evaluate.

values `array`

Array of `(expression, value)` substitution pairs. Values may be real numbers or `(re: ..., im: ...)` dictionaries.

Default: `()`

5.4.2 domain

Describe one real sampling axis for `evaluate-grid`.

Both endpoints are included when `samples` is greater than one. A single-sample domain evaluates only at `min`. The returned dictionary has the exact shape `(min: float, max: float, samples: int)`.

```
#repr(domain(-1, 1, samples: 3))
```

```
(min: -1.0, max: 1.0, samples: 3)
```

Parameters

```
domain(
  min: int float,
  max: int float,
  samples: int
) -> dictionary
```

min `int` or `float`

Finite lower endpoint; it must be less than `max`.

max `int` or `float`

Finite upper endpoint.

samples `int`

Number of evenly spaced points. Must be positive.

Default: `200`

5.4.3 evaluate-many

Evaluate one or more expressions at explicit points in one batch.

A single expression or variable may be passed directly; otherwise use an array. Every point must supply one real or complex value per variable. The result has one row per point in input order and one `(re: float, im: float)` dictionary per expression in expression order.

```
#let x = var("x")
#let y = var("y")
#let rows = evaluate-many((math($x + y$),
math($x y$)), (x, y), ((1, 2), (3, 4)))
#repr(rows)
```

```
(
  ((re: 3.0, im: 0.0), (re: 2.0, im: 0.0)),
  ((re: 7.0, im: 0.0), (re: 12.0, im: 0.0)),
)
```

Parameters

```
evaluate-many(
  expressions: bytes content array int float str,
  variables: bytes content array str,
  points: array
) -> array
```

expressions `bytes` or `content` or `array` or `int` or `float` or `str`

One expression or a non-empty array of expressions to evaluate.

variables `bytes` or `content` or `array` or `str`

One variable or an array defining input-column order.

points `array`

Array of input rows. Each row is an array ordered like `variables`; for one variable, a scalar point is also accepted.

5.4.4 evaluate-grid

Evaluate expressions over a Cartesian product of real domains in one batch.

The result is `(shape: array, points: array, values: array)`. `shape` lists the sample count of every domain. The grid axes are flattened into rows with the last domain varying fastest: each `points` row contains real coordinates in variable order, while the corresponding `values` row contains one `(re: float, im: float)` dictionary per expression.

```
#let x = var("x")
#let y = var("y")
#let grid = evaluate-grid(math($x^2 + y$), (x,
y), (domain(-1, 1, samples: 3), domain(0, 1,
samples: 2)))
shape: #repr(grid.shape); values:
#repr(grid.values)
```

```
shape: (3, 2) values: (
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
  ((re: 0.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
)
```

Parameters

```
evaluate-grid(
  expressions: bytes content array int float str,
  variables: bytes content array str,
  domains: array
) -> dictionary
```

expressions `bytes` or `content` or `array` or `int` or `float` or `str`

One expression or a non-empty array of expressions.

variables `bytes` or `content` or `array` or `str`

One variable or an array defining grid-axis order.

domains `array`

One `domain` dictionary per variable, in the same order.

5.4.5 solve-linear

Solve a linear system exactly for `variables`.

Each item in `system` is interpreted as an expression equal to zero. A vector matrix may be supplied instead of an array. The returned array is contains one atom payload per item in `variables`, in the same order. For an

underdetermined system, bound variables may be expressed using free variables; Symbolica chooses the highest-indexed variables as free. Inconsistent or nonlinear systems produce an error.

```
#let x = var("x")
#let y = var("y")
#let sol = solve-linear((math($2 x + y - 5$),
math($x - y - 1$)), (x, y))
#sol.map(to-typst).join[, ]
```

2, 1

Parameters

```
solve-linear(
  system: array bytes content int float str,
  variables: array content str
) -> array
```

system `array` or `bytes` or `content` or `int` or `float` or `str`

Linear expressions understood to equal zero, as an array or vector matrix.

variables `array` or `content` or `str`

Variables to solve for, in result order.

5.4.6 solve-system

Solve a linear or supported polynomial nonlinear system exactly.

Each expression in `system` is understood to equal zero. Polynomial systems are solved exactly through Symbolica's Gröbner-basis and algebraic-root machinery; coefficients may contain symbolic parameters when Symbolica can treat them rationally. The result is an array of solution rows. Every row contains one atom payload per requested variable, in `variables` order; an empty array means there are no solutions.

```
#let x = var("x")
#let y = var("y")
#let solutions = solve-system((math($x^2 - 1$),
math($y - x$)), (x, y))
#repr(solutions.map(row => row.map(canonical)))
```

(("1", "1"), ("-1", "-1"))

Parameters

```
solve-system(
  system: array content int float str,
  variables: array content str
) -> array
```

system `array` or `content` or `int` or `float` or `str`

Expressions understood to equal zero.

variables `array` or `content` or `str`

Variables to eliminate and return, in result-column order.

5.4.7 nsolve

Find a real root of a univariate expression with Newton's method.

The expression is interpreted as equal to zero and evaluated with `f64` arithmetic. Convergence is local and depends on `init`; failure to converge within `max-iterations` produces an error.

```
#let x = var("x")
#repr(nsolve(math($x^2 - 2$), x, 1.0))
```

```
1.4142135623746899
```

Parameters

```
nsolve(
  expr: bytes content int float str,
  var: bytes content str,
  init: int float,
  prec: int float,
  max-iterations: int
) -> float
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression understood to equal zero.

var `bytes` or `content` or `str`

Real solve variable, normally created with `var`.

init `int` or `float`

Initial real guess.

prec `int` or `float`

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations `int`

Maximum number of Newton iterations.

Default: `1000`

5.4.8 nsolve-system

Find a common real root of a system with multivariate Newton iteration.

Every expression is interpreted as equal to zero. `variables` and `init` must have matching lengths, and the returned floats follow that same order. Convergence is local and is not guaranteed for an arbitrary initial guess.

```
#let x = var("x")
#let y = var("y")
#repr(nsolve-system((math($x^2 + y - 3$),
math($x - y$)), (x, y), (1.0, 1.0)))
```

```
(1.3027756377319948, 1.3027756377319948)
```

Parameters

```
nsolve-system(
  system: array content int float str,
  variables: array content str,
  init: array,
  prec: int float,
  max-iterations: int
) -> array
```

system `array` or `content` or `int` or `float` or `str`

Expressions understood to equal zero.

variables `array` or `content` or `str`

Variables to solve for, in result order.

init `array`

Initial real value for every variable.

prec `int` or `float`

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations `int`

Maximum number of Newton iterations.

Default: `1000`

5.5 Matrices

5.5.1 matrix

Convert Typst values into an opaque Symbolica matrix payload.

Accepted forms are Typst math `mat(...)` or `vec(...)`, a non-empty rectangular nested array, a flat array interpreted as a column vector, a scalar interpreted as a one-by-one matrix, or an existing payload. Every entry must be convertible to a rational polynomial; general transcendental expressions are not valid matrix entries.

```
#to-typst(matrix($mat(1, 2; 3, 4)$))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix(value: bytes content array int float str) -> bytes
```

value `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix source value.

5.5.2 vec

Convert a non-empty sequence into a column-vector matrix payload.

Array entries must be rational-polynomial compatible. Typst math `vec(...)` is accepted directly; existing matrix bytes pass through unchanged.

```
#to-typst(vec((1, 2)))
```

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Parameters

`vec(values: array content bytes) -> bytes`

values `array` or `content` or `bytes`

Vector entries, Typst `vec(...)` content, or an existing matrix payload.

5.5.3 identity

Create an `n` by `n` identity matrix payload.

```
#to-typst(identity(2))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Parameters

`identity(n: int) -> bytes`

n `int`

Positive matrix dimension.

5.5.4 eye

Create a square diagonal matrix from a non-empty entry array.

Every entry must be rational-polynomial compatible.

```
#to-typst(eye((1, 2)))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

Parameters

`eye(diag: array) -> bytes`

diag `array`

Diagonal entries in top-left to bottom-right order.

5.5.5 matrix-add

Add two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-typst(matrix-add(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 2 & 2 \\ 3 & 5 \end{pmatrix}$$

Parameters

```
matrix-add(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix of the same shape.

5.5.6 matrix-sub

Subtract two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-typst(matrix-sub(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 0 & 2 \\ 3 & 3 \end{pmatrix}$$

Parameters

```
matrix-sub(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix of the same shape.

5.5.7 matrix-mul

Multiply two matrices, or multiply a matrix by a scalar expression.

Matrix multiplication requires compatible inner dimensions. A scalar must be rational-polynomial compatible. Because opaque bytes are interpreted as matrix payloads in the right-hand position, pass scalar atoms as ordinary values or math content rather than as preconstructed atom bytes.

```
#to-typst(matrix-mul(matrix(((1, 2), (3, 4))),
identity(2)))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-mul(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix, or scalar Typst value/math content.

5.5.8 matrix-div-scalar

Divide every matrix entry by a nonzero scalar expression.

The scalar must be rational-polynomial compatible.

```
#to-typst(matrix-div-scalar(matrix(((2, 4), (6, 8))), 2))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-div-scalar(
  lhs: bytes content array int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix dividend.

rhs `bytes` or `content` or `int` or `float` or `str`

Nonzero scalar divisor.

5.5.9 transpose

Transpose a matrix, exchanging rows and columns.

```
#to-typst(transpose(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Parameters

```
transpose(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to transpose.

5.5.10 det

Compute the exact determinant of a square matrix.

The result is an atom payload rather than a matrix payload.

```
#to-typst(det(matrix(((1, 2), (3, 4)))))
```

$$-2$$

Parameters

```
det(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Square matrix.

5.5.11 inv

Compute the exact inverse of an invertible square matrix.

Singular and non-square matrices produce an error.

```
#to-typst(inv(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$$

Parameters

```
inv(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Invertible square matrix.

5.5.12 matrix-solve

Solve the matrix equation `A x = b` exactly.

The row counts of `A` and `b` must agree. This strict solver reports an error when the system does not have the required unique solution; use `matrix-solve-any` for underdetermined systems.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Parameters

```
matrix-solve(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A `bytes` or `content` or `array` or `int` or `float` or `str`

Coefficient matrix.

b `bytes` or `content` or `array` or `int` or `float` or `str`

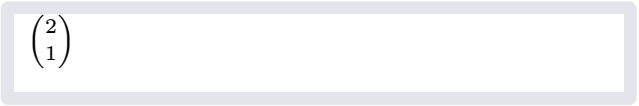
Right-hand-side matrix or column vector.

5.5.13 matrix-solve-any

Solve `A x = b` exactly, choosing one solution if underdetermined.

The row counts of `A` and `b` must agree. An inconsistent system still produces an error.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve-any(A, b))
```



Parameters

```
matrix-solve-any(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A `bytes` or `content` or `array` or `int` or `float` or `str`

Coefficient matrix.

b `bytes` or `content` or `array` or `int` or `float` or `str`

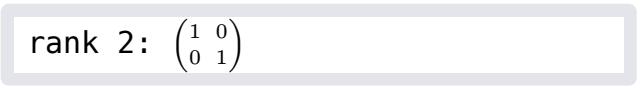
Right-hand-side matrix or column vector.

5.5.14 row-reduce

Row-reduce a matrix exactly using Gaussian elimination.

Returns `(matrix: bytes, rank: int)`. By default every column may contain a pivot. Set `max-col` to limit pivot search to the first `max-col` columns, which is useful for an augmented matrix whose trailing columns are right-hand sides; all columns are still transformed by the row operations.

```
#let rr = row-reduce(matrix(((1, 2), (3, 4))))
rank #rr.rank: #to-typst(rr.matrix)
```



Parameters

```
row-reduce(
  matrix: bytes content array int float str,
  max-col: int none
) -> dictionary
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to reduce.

max-col `int` or `none`

Number of leading columns eligible for pivots. `none` uses every column. An explicit value must be between zero and the column count.

Default: `none`

5.5.15 augment

Horizontally concatenate two matrices as `[lhs rhs]`.

Both matrices must have the same number of rows.

```
#to-typst(augment(identity(2), matrix(((1, 2),
(3, 4)))))
```

$$\begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 3 & 4 \end{pmatrix}$$

Parameters

```
augment(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left block.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right block with the same row count.

5.5.16 split-col

Split a matrix into left and right column blocks.

Returns exactly `(left, right)`, both as matrix payloads. The current matrix backend requires `index > 0` and `index < columns - 1`.

```
#let aug = augment(identity(2), matrix(((1, 2),
(3, 4))))
#let parts = split-col(aug, 2)
#to-typst(parts.at(0)) | #to-typst(parts.at(1))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \mid \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
split-col(
  matrix: bytes content array int float str,
  index: int
) -> array
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to split.

index `int`

Zero-based first column of the right block.

5.5.17 primitive-part

Divide a rational-polynomial matrix by its content.

The result is the primitive matrix whose coefficient GCD has been removed.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-typst(primitive-part(P))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
primitive-part(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix whose entries are rational polynomials.

5.5.18 content

Compute the coefficient content of a rational-polynomial matrix.

The content is the common coefficient GCD and is returned as an atom payload.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-typst(content(P))
```

$$2x$$

Parameters

```
content(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix whose entries are rational polynomials.

5.5.19 matrix-at

Read one matrix entry as an atom payload using zero-based indices.

```
#let A = matrix(((1, 2), (3, 4)))
#to-typst(matrix-at(A, 0, 1))
```

$$2$$

Parameters

```
matrix-at(
  matrix: bytes content array int float str,
  row: int,
  col: int
) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to index.

row `int`

Zero-based row index. Must be within the matrix bounds.

col `int`

Zero-based column index. Must be within the matrix bounds.

5.5.20 matrix-shape

Return the matrix shape as the two-element array `(rows, columns)`.

```
#repr(matrix-shape(matrix(((1, 2), (3, 4)))))
```

```
(2, 2)
```

Parameters

`matrix-shape(matrix: bytes content array int float str) -> array`

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

5.5.21 matrix-is-zero

Test exactly whether every matrix entry is zero.

This is an exact symbolic predicate, not a floating-point tolerance test.

```
#matrix-is-zero(matrix(((0, 0), (0, 0))))
```

```
true
```

Parameters

`matrix-is-zero(matrix: bytes content array int float str) -> bool`

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

5.5.22 matrix-is-diagonal

Test exactly whether every off-diagonal matrix entry is zero.

Rectangular matrices are accepted; entries outside the main diagonal must be zero.

```
#matrix-is-diagonal(matrix(((1, 0), (0, 2))))
```

```
true
```

Parameters

`matrix-is-diagonal(matrix: bytes content array int float str) -> bool`

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

5.5.23 matrix-derivative

Differentiate every matrix entry with respect to an indeterminate.

```
#let x = var("x")
#to-typst(matrix-derivative(matrix(((pow(x, 2), x), (1, 0))), x))
```

$$\begin{pmatrix} 2x & 1 \\ 0 & 0 \end{pmatrix}$$

Parameters

```
matrix-derivative(
  matrix: bytes content array int float str,
  var: bytes content str
) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Rational-polynomial matrix to differentiate.

var `bytes` or `content` or `str`

Indeterminate with respect to which each entry is differentiated.

5.6 Scalar constructors

5.6.1 add

Construct an exact sum from expression values.

Arguments are converted as by `atom`. With no arguments, the result is the additive identity zero.

```
#to-typst(add("x", 1, "y"))
```

$$1 + x + y$$

Parameters

```
add(..terms: arguments) -> bytes
```

..terms `arguments`

Positional terms to add.

5.6.2 mul

Construct an exact product from expression values.

Arguments are converted as by `atom`. With no arguments, the result is the multiplicative identity one.

```
#to-typst(mul(2, "x", "y"))
```

$$2xy$$

Parameters

```
mul(..factors: arguments) -> bytes
```

..factors `arguments`

Positional factors to multiply.

5.6.3 neg

Negate an expression exactly.

```
#to-typst(neg("x"))
```

$$-x$$

Parameters

```
neg(expr: bytes content int float str) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to negate.

5.6.4 sub

Subtract `rhs` from `lhs` exactly.

```
#to-typst(sub("x", "y"))
```

$$x - y$$

Parameters

```
sub(
  lhs: bytes content int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `int` or `float` or `str`

Minuend.

rhs `bytes` or `content` or `int` or `float` or `str`

Subtrahend.

5.6.5 div

Construct the exact quotient `lhs / rhs`.

```
#to-typst(div(1, "x"))
```

$$\frac{1}{x}$$

Parameters

```
div(
  lhs: bytes content int float str,
```

```
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `int` or `float` or `str`

Numerator.

rhs `bytes` or `content` or `int` or `float` or `str`

Denominator.

5.6.6 pow

Construct the exact power `base ^ exp`.

```
#to-typst(pow("x", 3))
```

x^3

Parameters

```
pow(
  base: bytes content int float str,
  exp: bytes content int float str
) -> bytes
```

base `bytes` or `content` or `int` or `float` or `str`

Base expression.

exp `bytes` or `content` or `int` or `float` or `str`

Exponent expression.

5.7 Advanced configuration

5.7.1 init

Create an independent set of Tymbolica functions.

The returned dictionary exposes Tymbolica's parsing, algebra, evaluation, solving, and matrix operations. The `"core"` profile uses the small plugin and leaves out symbolic integration. The `"full"` profile adds Rubi's `integrate` and `integrate-with-steps` methods. Use `init` when you want to select a profile, symbol namespace, plugin location, or parser grammar; ordinary calculations can use the imported top-level functions directly.

`integrate(expr, var)` returns Rubi's best-effort antiderivative as bytes. `integrate-with-steps(expr, var)` returns `(result: bytes, complete: bool, steps: array)`. Each step contains `rule` (int or none), `depth` (int), `description` (str), `references` (array of str), `source` (str), and the immediate `input` and `output` expressions as bytes. The steps run from an outer rewrite into its nested integrals. No integration constant is added.

Keep a calculation inside the engine that created its expressions. Core and full engines have separate Symbolica symbol registries, so their opaque atom bytes must not be exchanged.

```
#let sym = init(namespace: "physics")
#let v = sym.var
#let render = sym.canonical
#raw(render(v("x"), namespaces: true))
```

```
physics::x
```

Parameters

```
init(
  namespace: str,
  profile: str,
  source: str none,
  grammar: dictionary
) -> dictionary
```

namespace `str`

Default namespace for symbols parsed from Typst math or strings. A per-call `namespace` passed to `math` or `var` takes precedence.

Default: `"typst"`

profile `str`

Plugin profile. `"core"` selects the small Symbolica bundle; `"full"` selects the bundle with Rubi integration and genuine integration steps.

Default: `"core"`

source `str` or `none`

WebAssembly plugin path passed to Typst's `plugin` constructor. `none` selects the bundled plugin for `profile`. Relative custom paths are resolved by Typst from this source file.

Default: `none`

grammar `dictionary`

Parser grammar used by `math` and `array-tree` unless they receive an explicit override.

Default: `_default_grammar`

5.8 Full-profile integration methods

These methods are fields of the dictionary returned by `init(profile: "full")`; they are not imported as top-level functions. Bind them before use, as in the worked integration example, and keep their Atom inputs and outputs on the same full engine.

```
integrate(expr, var) -> bytes
integrate-with-steps(expr, var) -> dictionary
```

`integrate` returns Rubi's best antiderivative without adding $+C$. An unfinished result contains an `unintegrable` marker. `integrate-with-steps` returns the same result together with:

- `result` (`bytes`): the best antiderivative;
- `complete` (`bool`): whether Rubi finished every subintegral;
- `steps` (`array`): the ordered transformation tree.

Each step contains `rule` (`int` or `none`), `depth` (`int`), `description` (`str`), `references` (`array` of strings), `source` (`str`), and the Atom payloads `input` and `output` (`bytes`). Steps run from an outer rewrite into the nested integrals it creates; `rule: none` marks an auxiliary transformation such as a fresh-symbol substitution.

6 Compatibility and licensing

This manual describes Tymbolica 0.1.0. The package is tested with Typst 0.14 or newer.

Tymbolica's original source code is released under the MIT License. Symbolica is developed by the Symbolica contributors and is distributed under its own license terms. The MIT License does not relicense Symbolica or either bundled WebAssembly artifact; Symbolica's terms still apply to their use. The full plugin's symbolic integration is supplied by the MIT-licensed `symbolica-integrate` crate and its port of the Rubi rules.

For source, issues, and release history, visit github.com/lcnbr/tymbolica.

7 Acknowledgements

Tymbolica would not exist without Symbolica. Thank you to its contributors for building and sharing the algebra engine at the heart of this package, and for making Rubi integration available through `symbolica-integrate`. Thanks as well to the Rubi contributors whose rule collection powers the full integration plugin.

Thanks also to Parsely, which makes it possible to work with mathematics written directly in Typst, and to Tidy, which powers this manual's examples and reference pages.